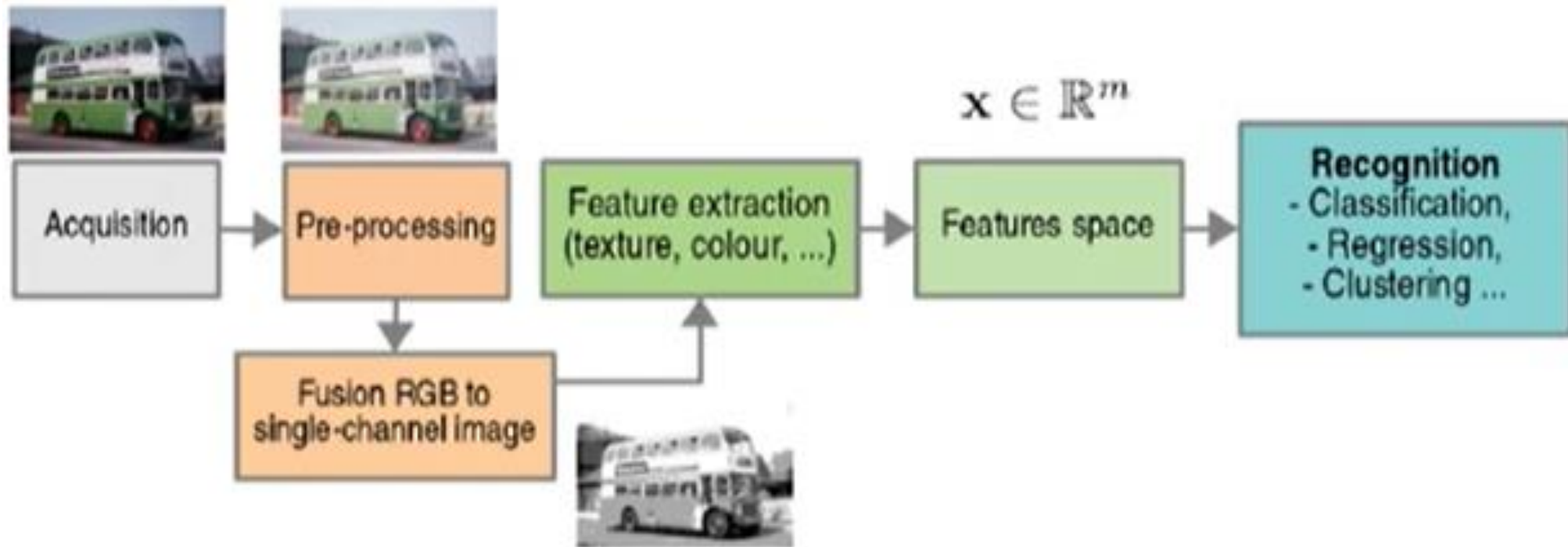


Redes Neurais para Classificação de Imagens

Guillermo Cámara Chávez

Pipeline classico para reconhecimento de imagens



Pipeline classico para reconhecimento de imagens

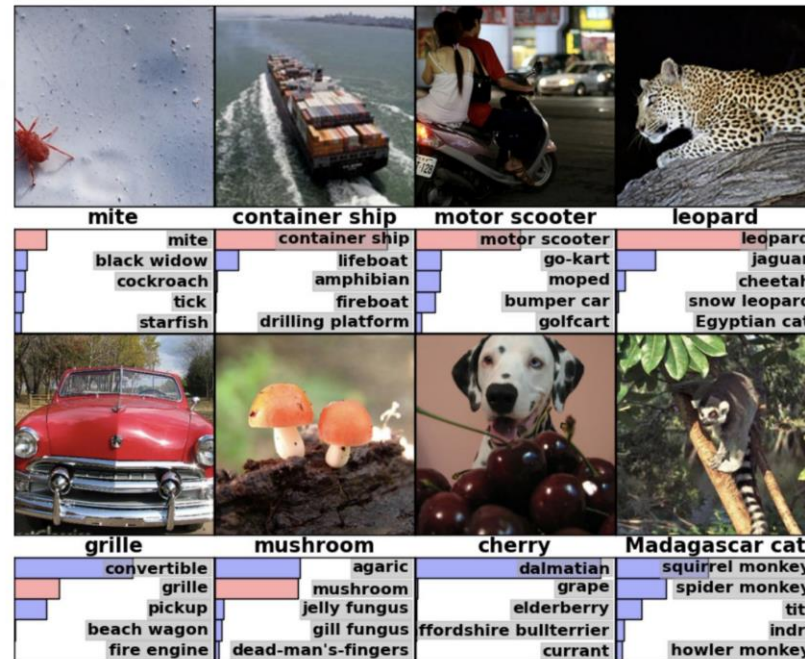
- Historia de métodos
 - Descritores de cor, forma e textura (1970 – 2000)
 - Scale invariant features – SIFT (> 1999)
 - Histograma de gradientes orientados (> 2005)
 - *Bag of Features* (> 2004)
 - Spatial Pyramid Matching (> 2006)

Avanço: dados anotados disponíveis

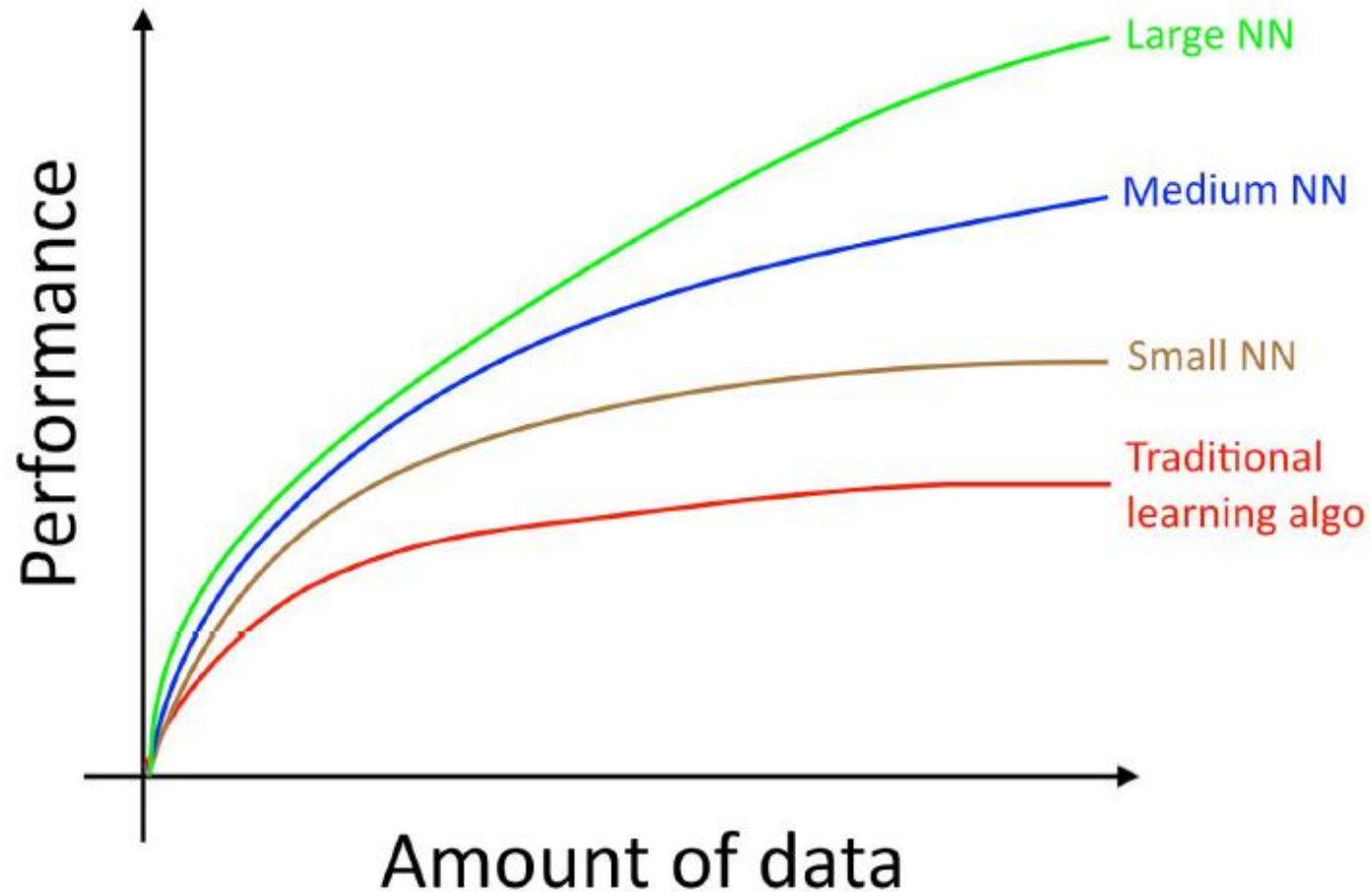
ImageNet Challenge

IMAGENET

- 1,000 object classes (categories).
- Images:
 - 1.2 M train
 - 100k test.

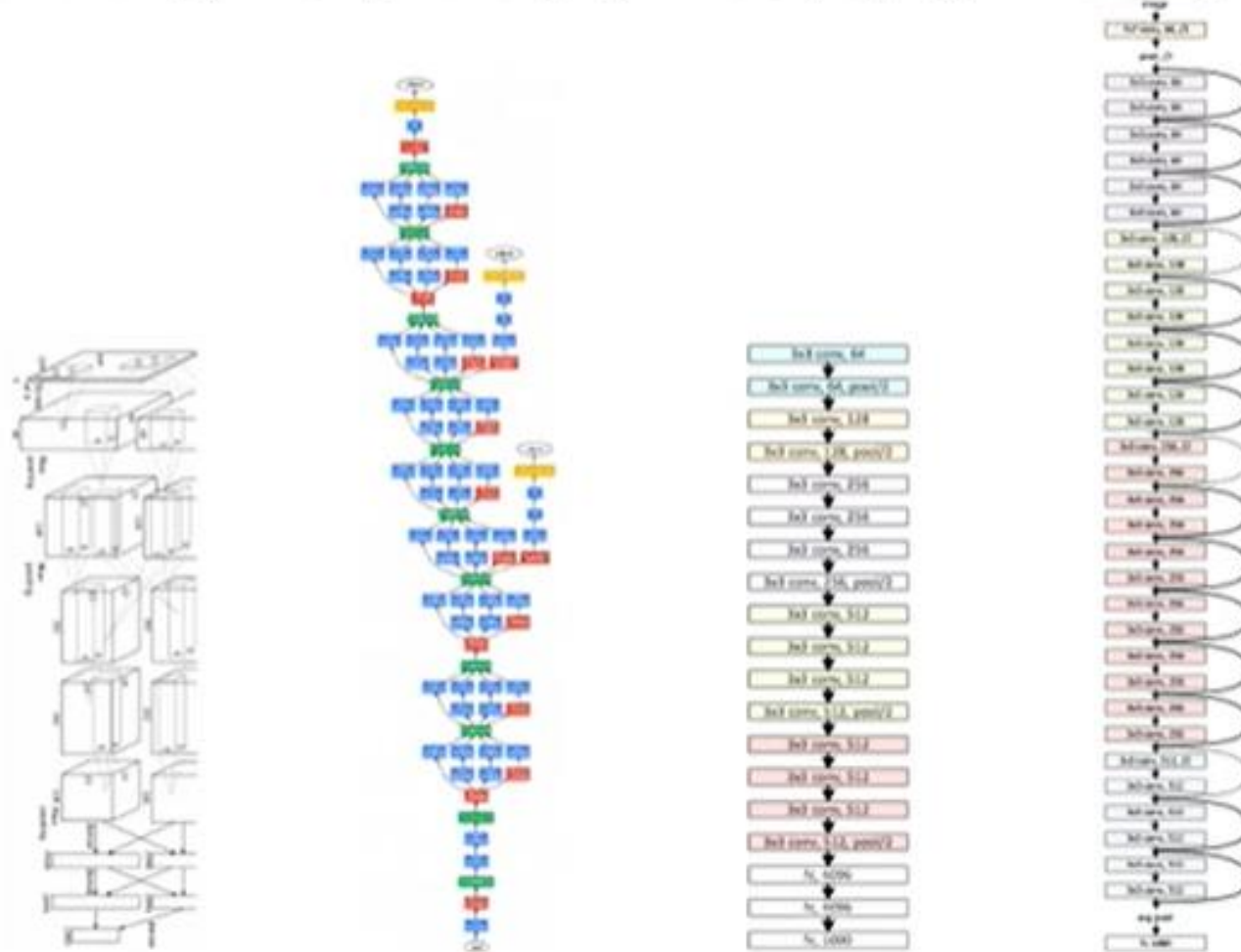


Avanço: dados anotados disponíveis



Redes neurais profundas assumem a classificação de imagens

AlexNet (9) GoogLeNet (22) VGG (16/19) ResNet (34+)



Handcrafted features

Manual Feature Extraction

Domain knowledge

Define features

Detect features
to classify

Viewpoint variation



Scale variation



Deformation



Occlusion



Illumination conditions



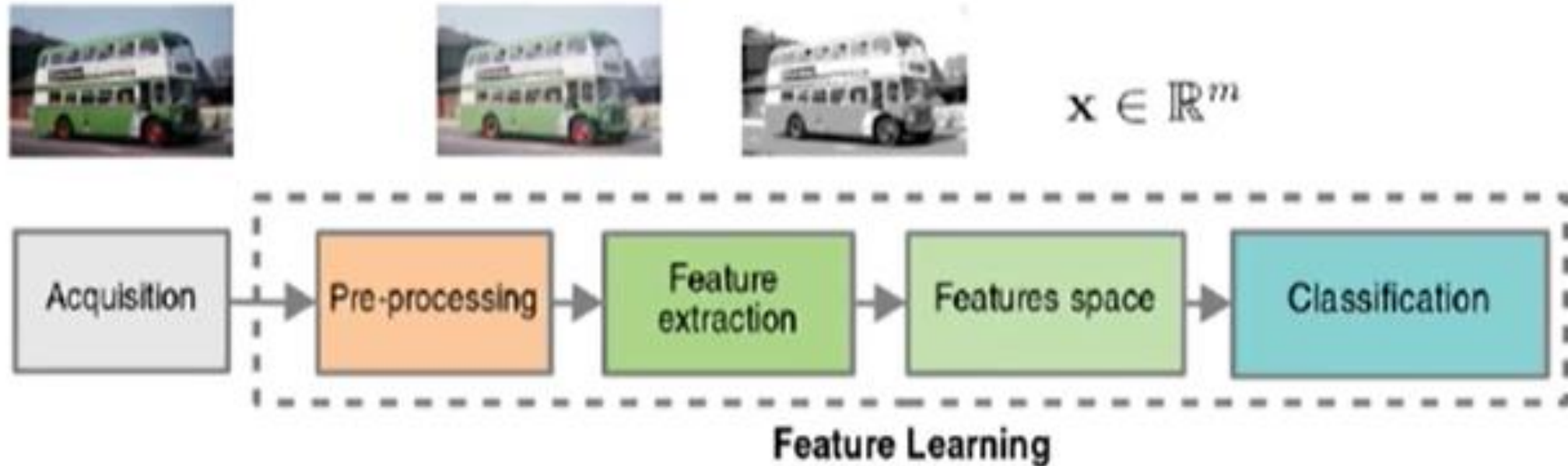
Background clutter



Intra-class variation



Novo pipeline: aprendizado de características



TRADITIONAL MACHINE LEARNING



DEEP LEARNING



O que é Deep Learning?

What is Deep Learning?

ARTIFICIAL INTELLIGENCE

Any technique that enables computers to mimic human behavior



MACHINE LEARNING

Ability to learn without explicitly being programmed



DEEP LEARNING

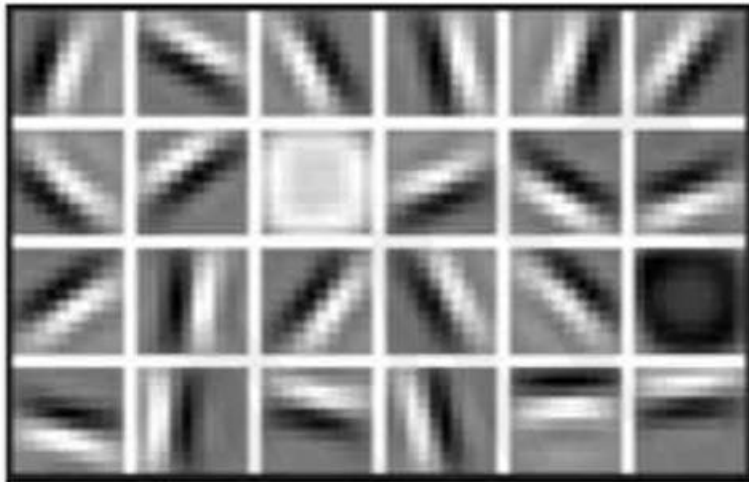
Extract patterns from data using neural networks



Por que Deep Learning?

- Recursos projetados manualmente são demorados e não são escaláveis na prática.
- Podemos aprender características diretamente dos dados?

Low Level Features



Lines & Edges

Mid Level Features



Eyes & Nose & Ears

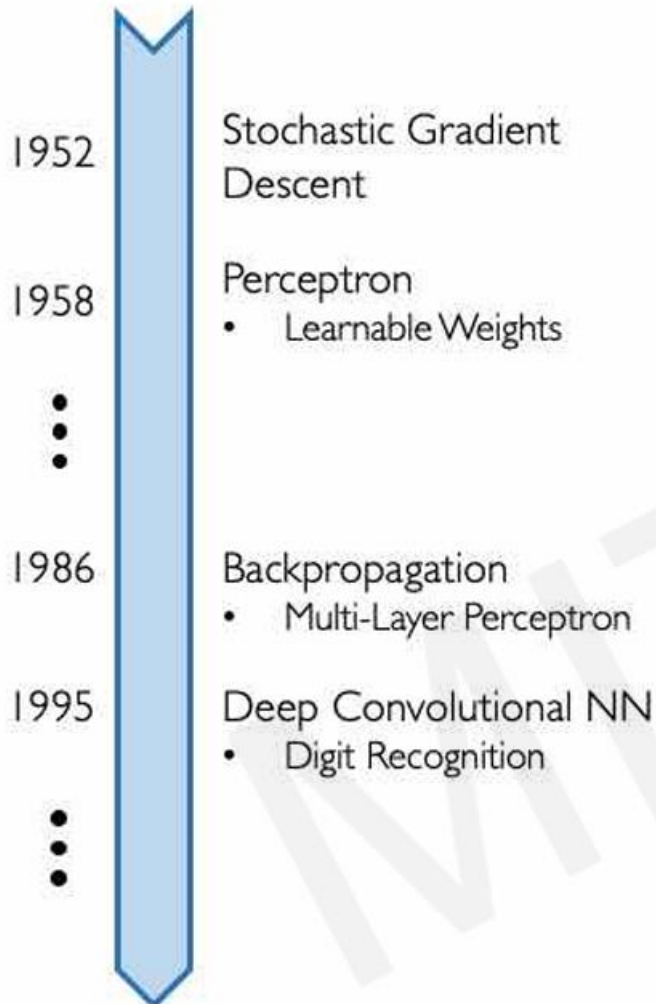
High Level Features



Facial Structure

Por qué agora?

- Os dados das redes neurais remontam a décadas, então por que o ressurgimento?



1. Big Data

- Larger Datasets
- Easier Collection & Storage

IMAGENET



2. Hardware

- Graphics Processing Units (GPUs)
- Massively Parallelizable



3. Software

- Improved Techniques
- New Models
- Toolboxes



Aprendizado de máquina - Aplicação

- Encontrar uma função na forma $f(\mathbf{x})=\mathbf{y}$, o significado delas depende da tarefa
- Classificação de imagens:
 - Data disponível: par (imagens e etiquetas) de deserto, praia e montanhas
 - Entrada: imagem RGB na forma de $\mathbf{x}$
 - Saída: etiqueta prevista $\mathbf{y}$ (por exemplo, montanha) atribuído à imagem de entrada

Machine Learning vs Deep Learning

- Machine learning
 - Uma area mais extensa que inclui DL
 - Algoritmos visa inferir $f: X \rightarrow Y$ de um espaço de funções admissíveis f dado um conjunto de treinamento
 - Métodos superficiais (*shallow*) geralmente inferem uma única função $f(.)$.
E.g. uma função linear $f(x) = w.x + b$
 - Os algoritmos são responsáveis de encontrar os valores de w e b
 - Exemplos: Perceptron, SVM, classificador de regressão logística

Machine Learning vs Deep Learning

- Deep Learning

- Envolve aprender uma sequência de representações por meio de funções compostas
- Dada uma entrada x_1 , várias representações intermediárias são produzidas.

- $x_2 = f_1(x_1), \quad x_3 = f_2(x_2), \quad x_4 = f_3(x_3), \dots$

- A saída depende de L funções aninhadas na forma de:

$$f_L(\dots f_3(f_2(f_1(x_1, \Theta_1) \Theta_2) \Theta_3 \dots, \Theta_L))$$

- Θ_i são parâmetros associados a cada função i

Shallow linear classifier



- Θ contem a matriz de pesos W e o vetor b de bias

$$\begin{aligned} f(\Theta, \mathbf{x}) &= \begin{array}{c} \text{weight} \\ \text{matrix} \end{array} \begin{array}{c} | \\ \mathbf{W} \end{array} \begin{array}{c} \text{image} \\ | \\ \mathbf{x} \end{array} + \begin{array}{c} \text{bias} \\ \text{term} \\ | \\ \mathbf{b} \end{array} \\ &= \text{scores for possible classes of } \mathbf{x} \end{aligned}$$

Classificador linear para classificação de imagens

- Entrada: imagem ($N \times M \times 3$) vetorizado em uma coluna x
- Classes: gato, tartaruga, coruja
- Saída: *scores* das classes



$$= x = [1, 73, 227, 82]$$

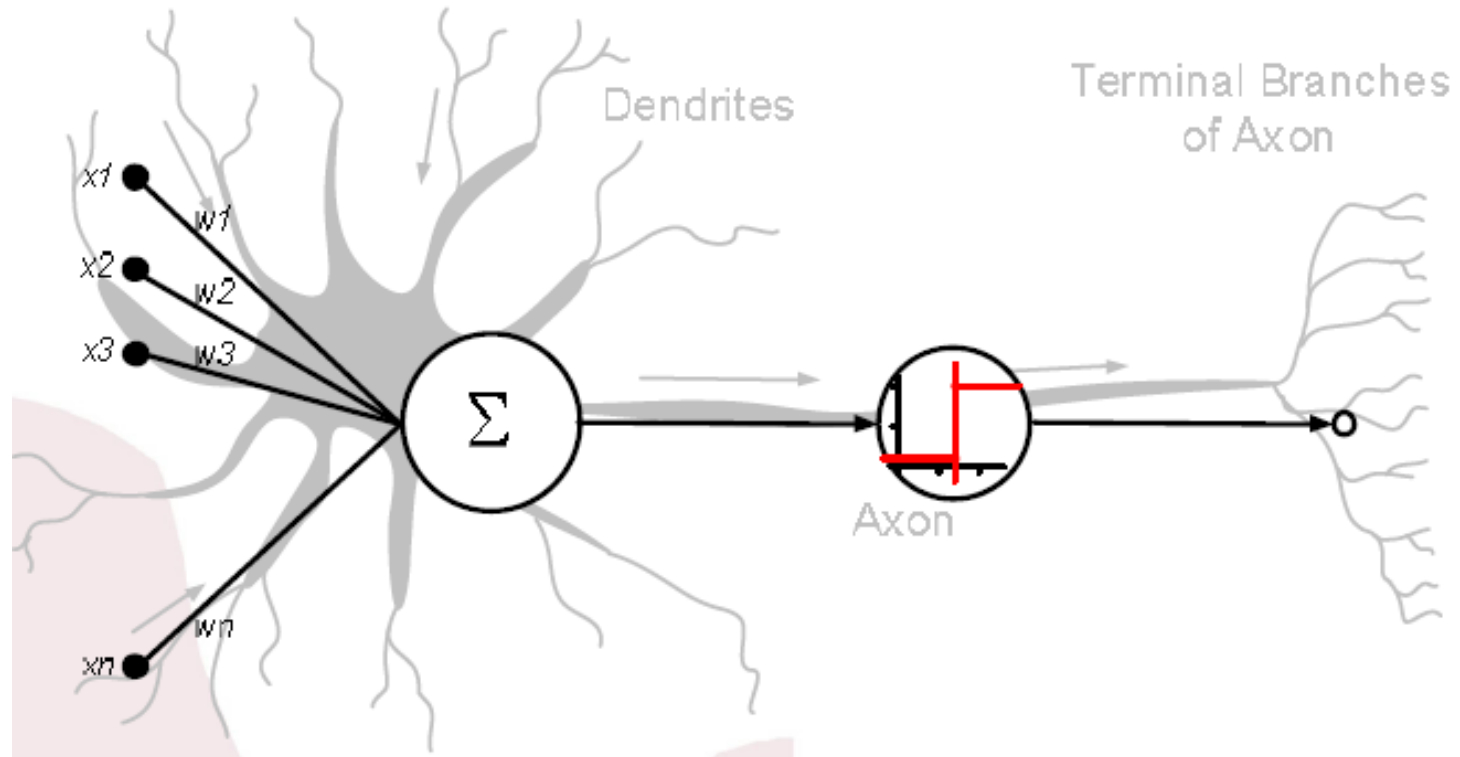
$f(\Theta, x) = s \rightarrow$ 3 numbers with class scores

$$\begin{bmatrix} 0.1 & -0.25 & 0.1 & 2.5 \\ 0 & 0.5 & 0.2 & -0.6 \\ 2 & 0.8 & 1.8 & -0.1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 73 \\ 227 \\ 82 \end{bmatrix} + \begin{bmatrix} -2.0 \\ 1.7 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -337.3 \\ -38.6 \\ 460.30 \end{bmatrix}$$

Perceptron

O bloco de construção estrutural do aprendizado profundo

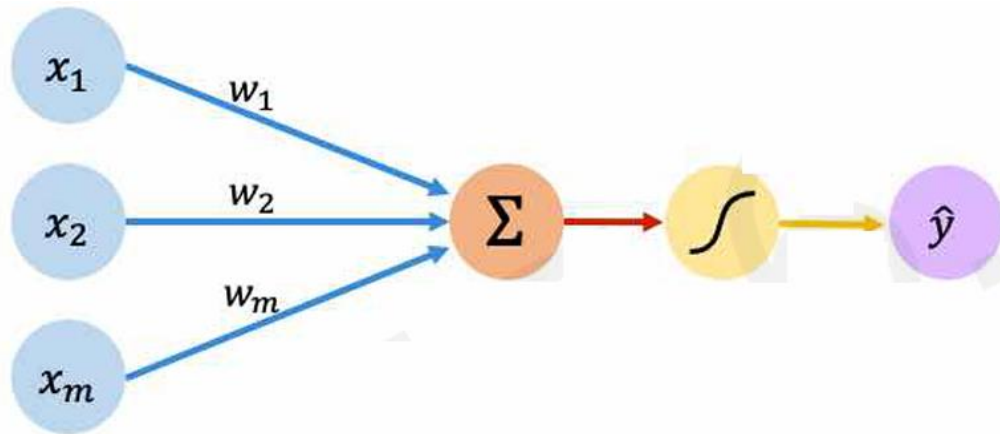
- Funções de classificação binária
- Função de ativação



Perceptron

O bloco de construção estrutural do aprendizado profundo

- Forward propagation



Entrada Pesos Soma Não-Lineal Saída

$$\hat{y} = g\left(\sum_{i=1}^m x_i w_i + b\right)$$

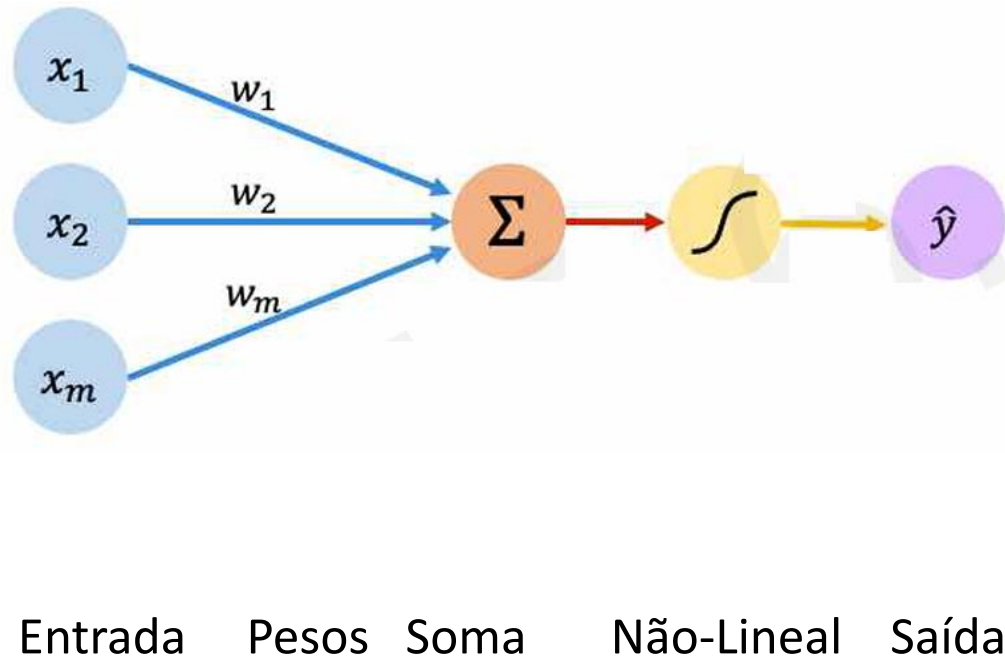
Diagram illustrating the mathematical formula for the output of a perceptron during forward propagation:

- $\hat{y}$: Output
- g : Função de ativação não linear (Non-linear activation function)
- $\sum_{i=1}^m x_i w_i$: Combinação linear (Linear combination)
- b : Bias

Perceptron

O bloco de construção estrutural do aprendizado profundo

- Forward propagation



$$\hat{y} = g\left(\sum_{i=1}^m x_i w_i + b\right)$$

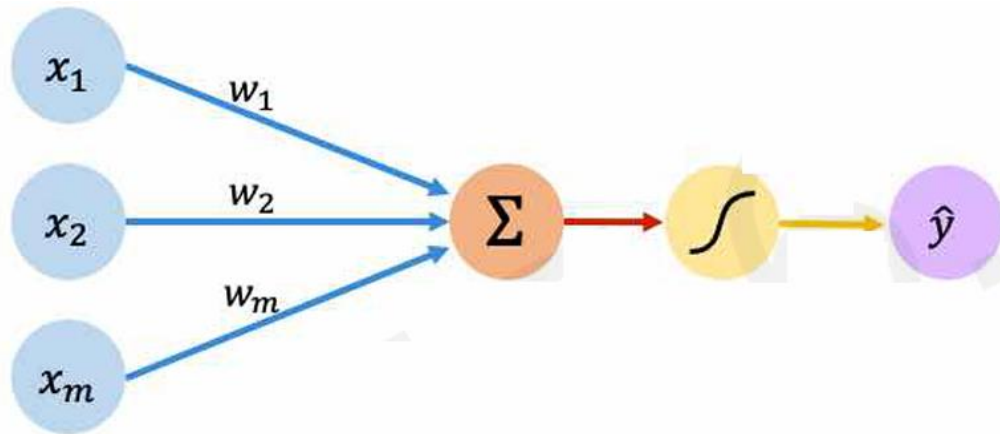
$$\hat{y} = g(x^T W + b)$$

$$X = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \quad W = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

Perceptron

O bloco de construção estrutural do aprendizado profundo

- Forward propagation

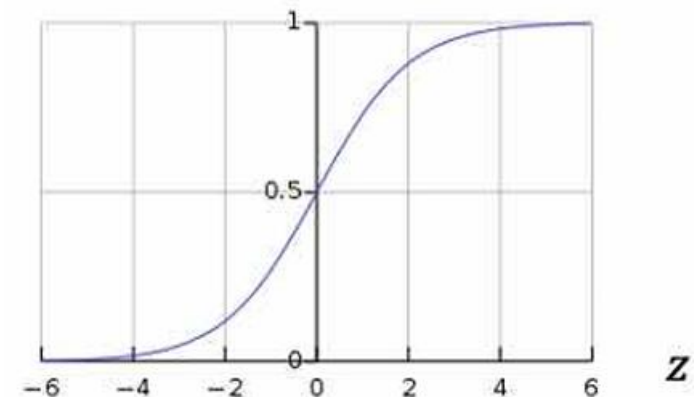


Entrada Pesos Soma Não-Lineal Saída

$$\hat{y} = g\left(\sum_{i=1}^m x_i w_i + b\right)$$

Exemplo: Função sigmoide

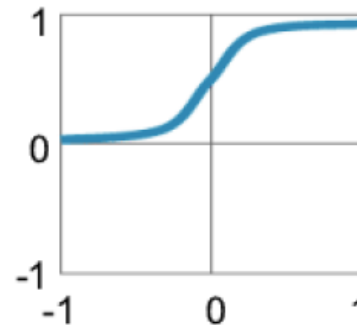
$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



Funções de ativação

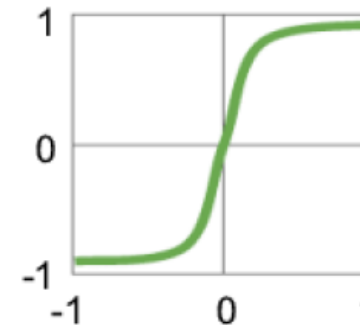
Traditional Non-Linear Activation Functions

Sigmoid



$$y = 1 / (1 + e^{-x})$$

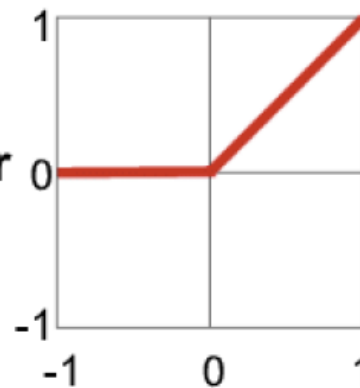
Hyperbolic Tangent



$$y = (e^x - e^{-x}) / (e^x + e^{-x})$$

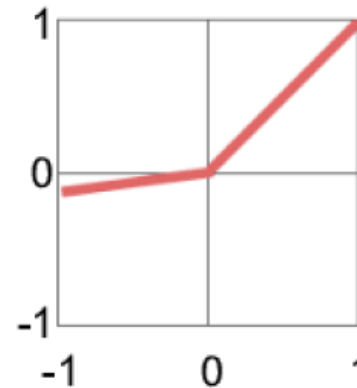
Modern Non-Linear Activation Functions

Rectified Linear Unit
(ReLU)



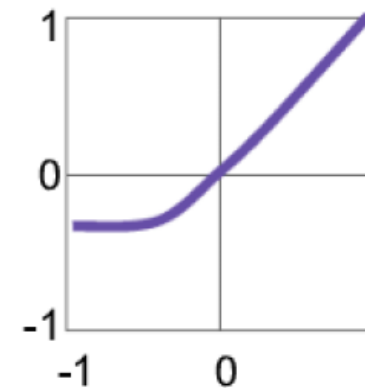
$$y = \max(0, x)$$

Leaky ReLU



$$y = \max(\alpha x, x)$$

Exponential LU



$$y = \begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$$

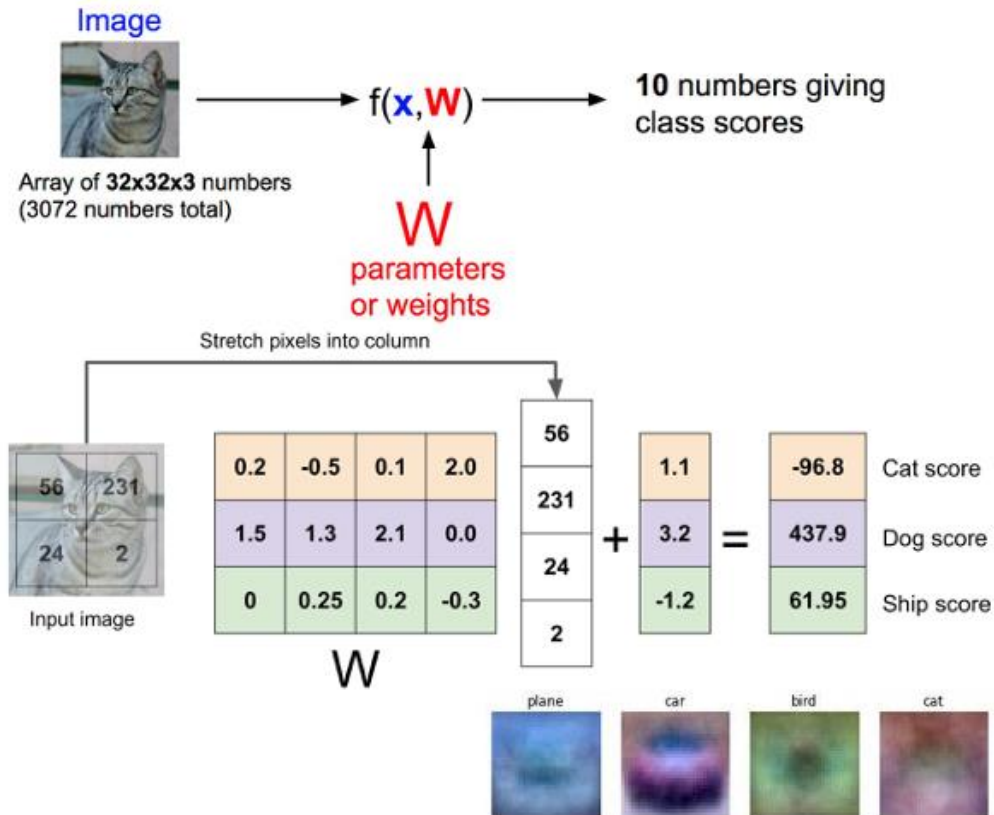
α = small const. (e.g. 0.1)

ReLU

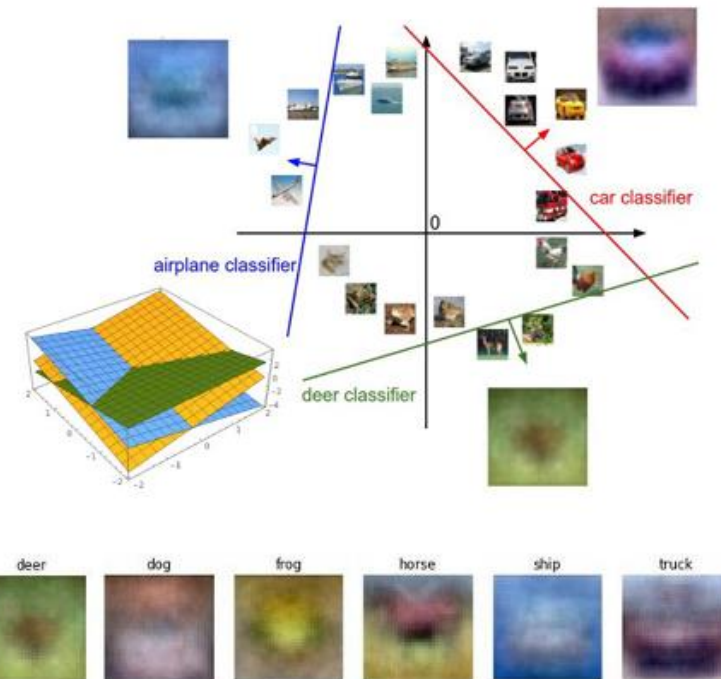
- *Rectified Linear Units*
- Função de ativação comumente utilizada após uma camada de convolução
- Objetivo: introduzir não linearidade
- Utilizada no lugar de *tanh* e *sigmoid* pois treina mais rápido e não interfere muito na acurácia
- Aplica a função $f(x) = \max(0, x)$
 - Transforma valores negativos em zero

Classificador linear

Recall from last time: Linear Classifier



$$f(x, W) = Wx + b$$



Classificador linear



- Defina uma função de perda (*Loss function*) que quantifique nossa infelicidade com as pontuações nos dados de treinamento.
- Crie uma maneira de encontrar com eficiência os parâmetros que minimizam a função de perda (otimização)

airplane	-3.45	-0.51	3.42
automobile	-8.87	6.04	4.64
bird	0.09	5.31	2.65
cat	2.9	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	-4.34
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

Classificador linear

- *Loss Function* (função de perda)
 - Uma função de perda informa o quão bom é nosso classificador atual
 - Valor baixo = bom classificador
 - Valor alto = péssimo classificador
 - Também é chamado de: função objetivo, função de custo

Classificador linear

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

A **loss function** tells how good our current classifier is

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^N$$

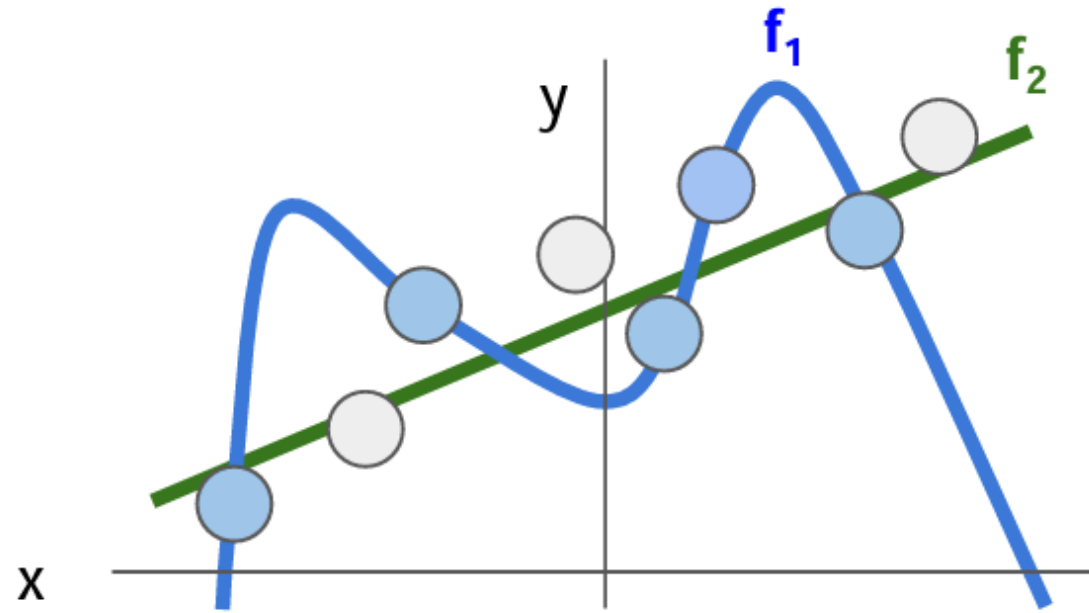
Where x_i is image and
 y_i is (integer) label

Loss over the dataset is a
sum of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Intuição de regularização

Regularization: Prefer Simpler Models



Regularization pushes against fitting the data *too* well so we don't fit noise in the data

Classificador linear

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss: Model predictions should match training data}} + \underbrace{\lambda R(W)}_{\text{Regularization: Model should be "simple", so it works on test data}}$$

Data loss: Model predictions should match training data

Regularization: Model should be “simple”, so it works on test data

Simple examples

L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

More complex:

Dropout

Batch normalization

Cutout, Mixup, Stochastic depth, etc...

Classificador linear

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss: Model predictions should match training data}} + \underbrace{\lambda R(W)}_{\text{Regularization: Model should be "simple", so it works on test data}}$$

Data loss: Model predictions should match training data

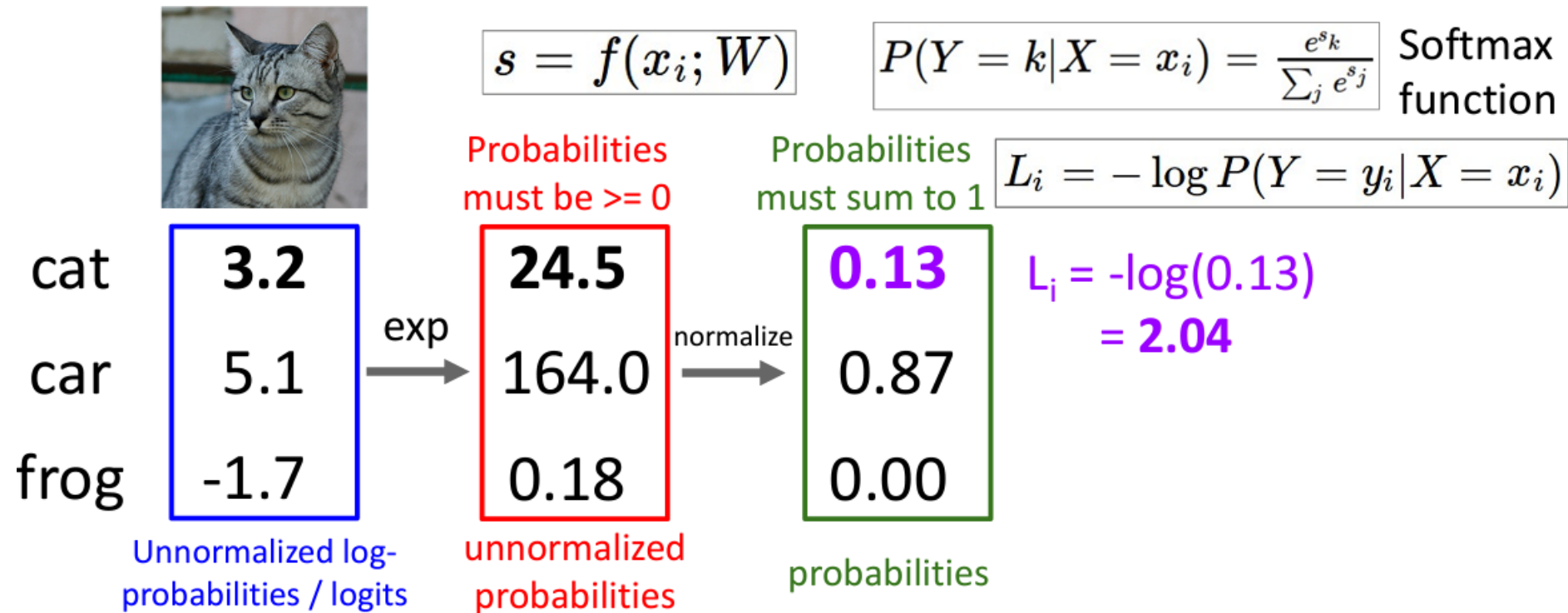
Regularization: Model should be “simple”, so it works on test data

Finalidade da regularização

- expressar preferências entre os modelos além de "minimizar o erro de treinamento"
- evite *overfitting*: prefira modelos simples que generalizem melhor

Classificador linear

- *Cross-Entropy Loss*
 - Interpretar a saída do classificador como probabilidades



Como minimizar a função de perda?

Otimização



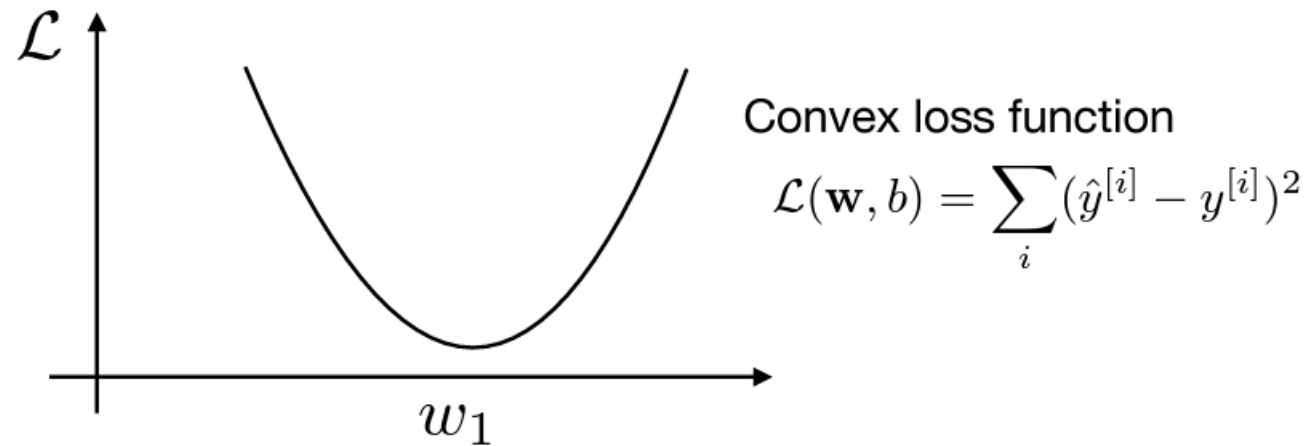
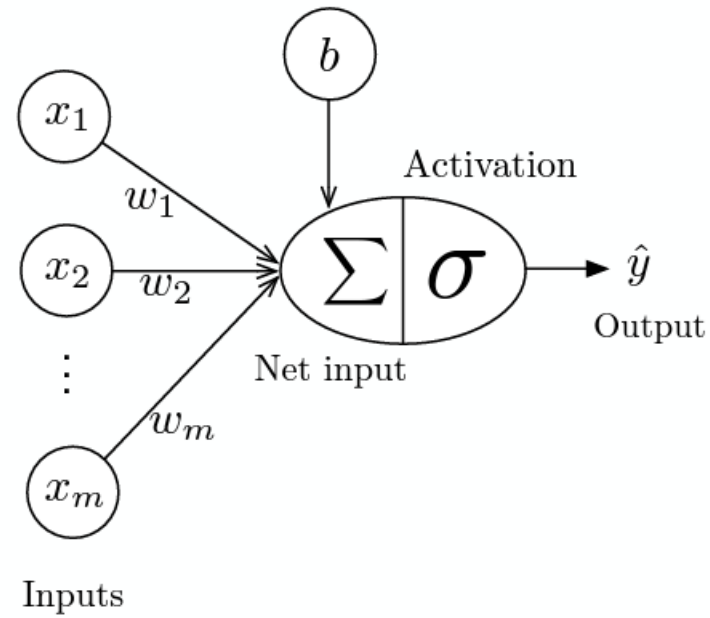
Como minimizar a função de perda?

Otimização

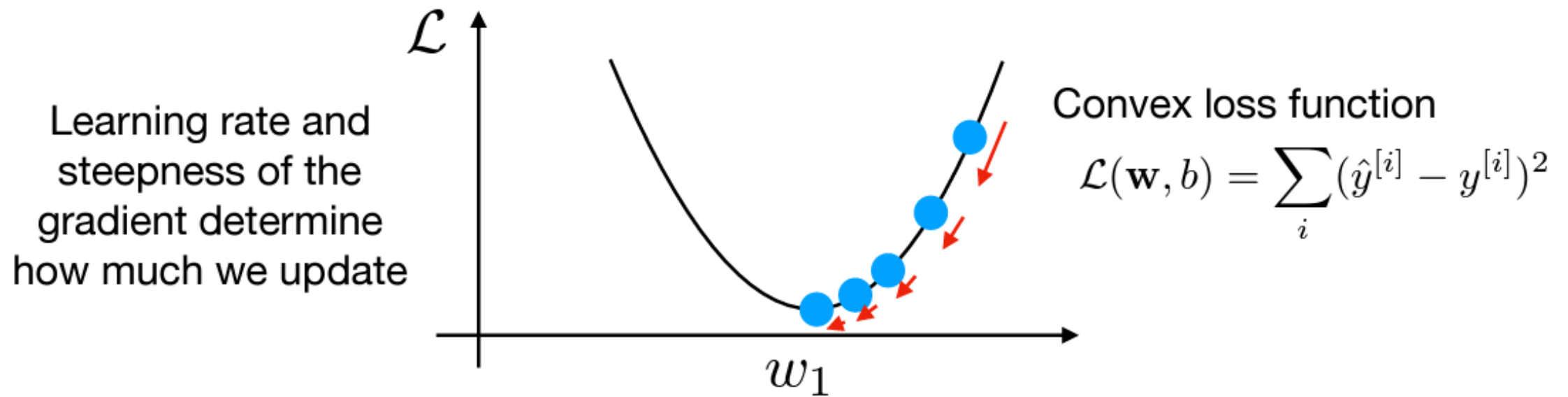


Seguir o
declive

Otimização

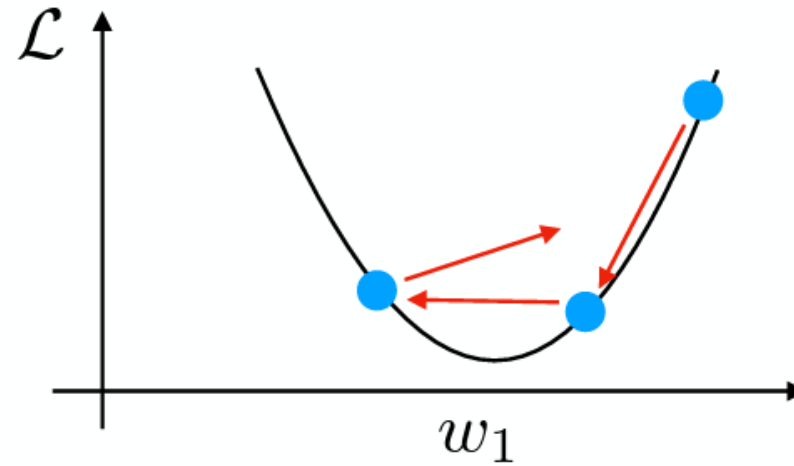


Gradient descent

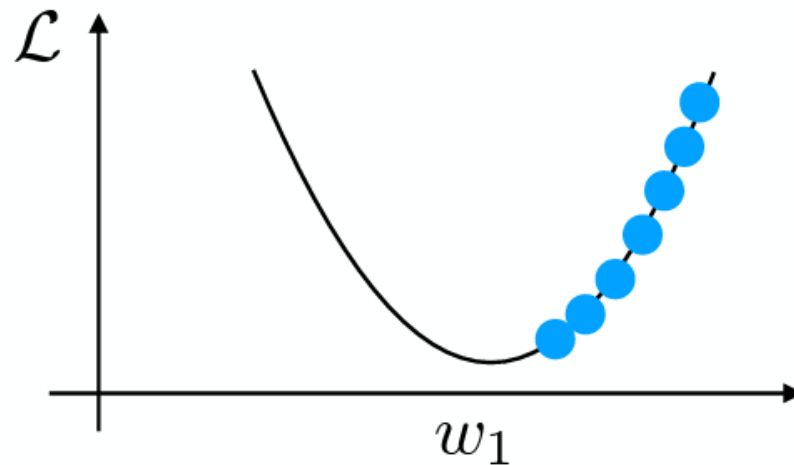


Gradient descent

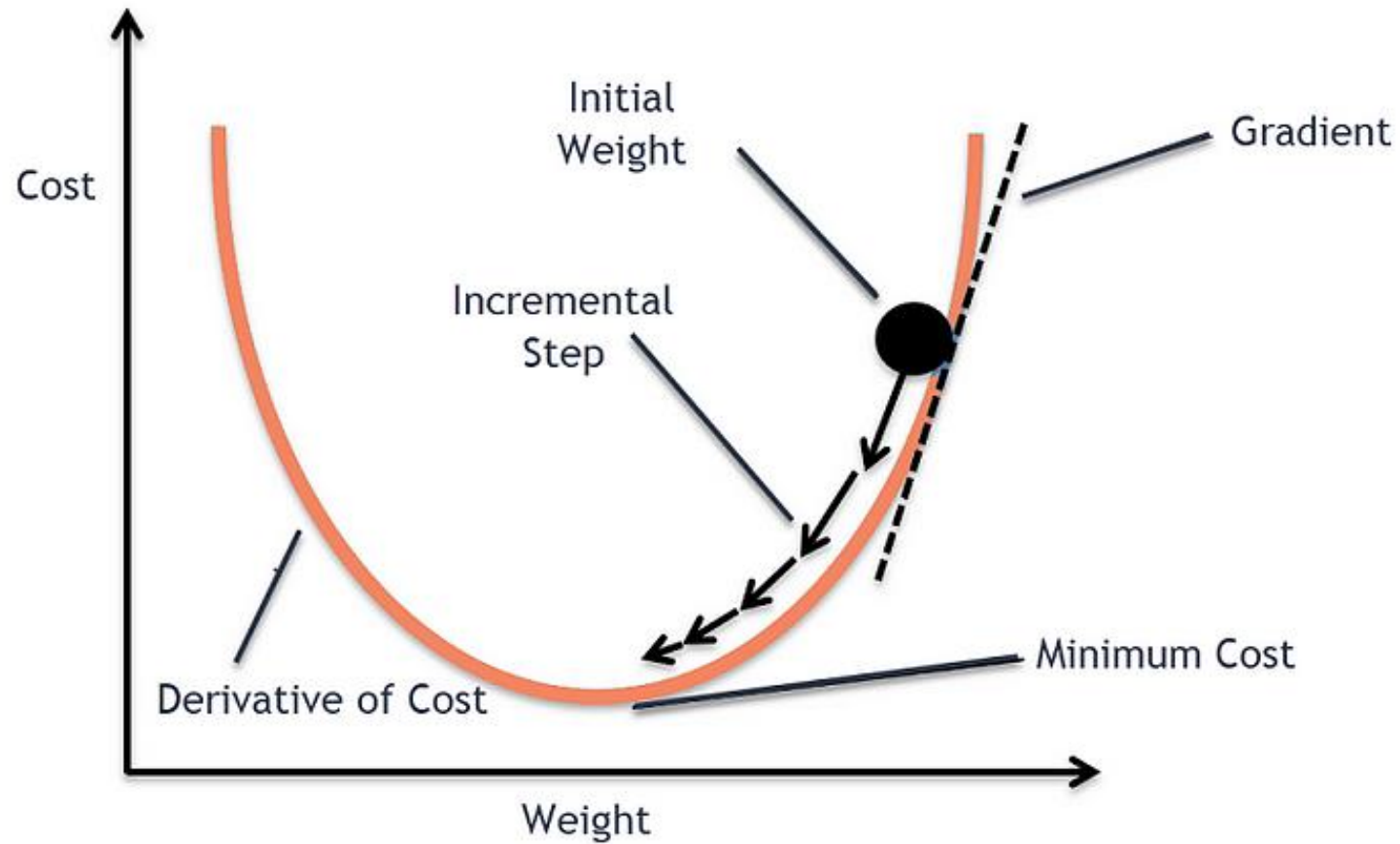
If the learning rate is too large,
we can overshoot



If the learning rate is too small,
convergence is very slow



Gradient descent



Gradient descent

Perceptron learning rule

1. Initialize $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m$, $b := 0$
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$
 - (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$
 - (b) $err := (y^{[i]} - \hat{y}^{[i]})$
 - (c) $\mathbf{w} := \mathbf{w} + err \times \mathbf{x}^{[i]}$
 $b := b + err$

Stochastic gradient descent

1. Initialize $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m$, $b := 0$
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$
 - (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$
 - (b) $\nabla_{\mathbf{w}} \mathcal{L} = (y^{[i]} - \hat{y}^{[i]}) \mathbf{x}^{[i]}$
 $\nabla_b \mathcal{L} = (y^{[i]} - \hat{y}^{[i]})$
 - (c) $\mathbf{w} := \mathbf{w} + \eta \times (-\nabla_{\mathbf{w}} \mathcal{L})$
 $b := b + \eta \times \underbrace{(-\nabla_b \mathcal{L})}_{\substack{\uparrow \\ \text{negative gradient}}}$


learning rate


negative gradient

Algoritmos de otimização

- *Gradient descent*
 - *Batch gradient descent*: todos os dados de treinamento são levados em consideração para dar um único passo
 - *Stochastic gradient descent (SGD)*: considere apenas um exemplo de cada vez para dar um único passo.
 - *Mini batch gradient descent*: uma mistura de *Batch Gradient Descent* e *SGD* é usada
 - SGD com momentum
 - Adaptive Gradient Algorithm (Adagrad)
 - RMSprop
 - Adaptive Moment Estimation (Adam)